

A PROJECT REPORT ON

**DEFENCE ASSET TRACKING &
GEOFENCING SYSTEM**

*Real-Time Telemetry Tracking, Geographic Perimeter Security, and Alerts Ingestion using
PostGIS & GeoServer Integration*

Submitted in partial fulfillment of the requirements for the award of

INTERNSHIP COMPLETION CERTIFICATE

Developed during the DRDO Internship Program (May – July 2026)

By a Collaborative Student Internship Team:

- Mahendra Gurjar (3rd Year, CSE)
- Priyadarshini Choudhary (3rd Year, IT)
 - Shahina Parvin (3rd Year, CSE)
 - Gaurav Deora (2nd Year, CSE)
 - Omprakash (2nd Year, CSE)
- Chandrika Solanki (2nd Year, IT)
- Abhimanyu Singh Rajpurohit (2nd Year, IT)
 - Pinku Daila (2nd Year, AIDS)
- Kulwant Singh Rathore (2nd Year, ECC)

Under the Guidance of:

Shri Shyam Lal
Scientist 'F' / Internship Mentor
Defence Laboratory Jodhpur (DLJ)

DEFENCE LABORATORY JODHPUR (DLJ)
DEFENCE RESEARCH & DEVELOPMENT ORGANISATION (DRDO)
MINISTRY OF DEFENCE, GOVERNMENT OF INDIA
JODHPUR, RAJASTHAN – 342011

DEFENCE LABORATORY JODHPUR (DLJ)

DEFENCE RESEARCH & DEVELOPMENT ORGANISATION (DRDO)

JODHPUR, RAJASTHAN

CERTIFICATE OF ORIGINALITY

This is to certify that the project report entitled "Defence Asset Tracking & Geofencing System" is a bona fide record of work carried out by the Student Internship Team under the supervision and guidance of Shri Shyam Lal, Scientist 'F' at Defence Laboratory Jodhpur (DLJ), Jodhpur, during the period from May 2026 to July 2026, as part of their internship program. The work presented is original and has not been submitted elsewhere for the award of any degree or diploma.

Shri Shyam Lal
Scientist 'F' / Project Coordinator
Defence Laboratory Jodhpur (DLJ)
DRDO, Jodhpur

DECLARATION BY STUDENTS

We, the members of the Student Internship Team, hereby declare that the project work presented in this report entitled "Defence Asset Tracking & Geofencing System" is our own work, carried out under the guidance of Shri Shyam Lal, Scientist 'F', Defence Laboratory Jodhpur (DLJ). We have fully acknowledged all sources of information and references utilized during this project.

Names and Signatures of Team Members:

_____	Mahendra Gurjar (3rd Year, CSE)
_____	Priyadarshini Choudhary (3rd Year, IT)
_____	Shahina Parvin (3rd Year, CSE)
_____	Gaurav Deora (2nd Year, CSE)
_____	Omprakash (2nd Year, CSE)
_____	Chandrika Solanki (2nd Year, IT)
_____	Abhimanyu Singh Rajpurohit (2nd Year, IT)
_____	Pinku Daila (2nd Year, AIDS)
_____	Kulwant Singh Rathore (2nd Year, ECC)

ACKNOWLEDGEMENT

First and foremost, we express our profound gratitude to the Defence Research and Development Organisation (DRDO) and especially Defence Laboratory Jodhpur (DLJ) for providing this invaluable learning opportunity and a highly professional research environment. It was an honor to develop a real-world GIS-based defence logistics system under their supervision.

We convey our deepest appreciation and respect to our Internship Mentor, Shri Shyam Lal, Scientist 'F', DLJ. His expert technical guidance, encouraging insights, and rigorous architecture review sessions were instrumental throughout the project lifecycle. His mentoring taught us the value of system resilience, geographic coordinate conventions, database indexing, and enterprise-grade Docker deployments.

We also acknowledge the academic departments of our respective universities for supporting our internship participation and facilitating our technical preparation. Finally, we thank our peers and fellow interns at DLJ for their collaboration, peer reviews, and creative inputs during the design of our system.

Date: July 2026

Place: Jodhpur

ABSTRACT

The "Defence Asset Tracking & Geofencing System" is a production-grade, real-time spatial asset monitoring, alert generation, and perimeter geofencing management application designed for defence sector logistics and perimeter security. The main challenge addressed by the system is the dynamic computing of high-frequency GPS coordinate streams against restricted zone shapes, ensuring low-latency alerts on unauthorized entries or exits. The system decouples presentation from geometric calculations by employing a standard three-tier architecture: a lightweight dark-themed SPA client powered by Leaflet.js; a robust Java 17 Servlet backend executing on Apache Tomcat 9; and a PostgreSQL database extended with PostGIS. Real-time GIS rendering is powered by GeoServer via WMS (Web Map Service) and WFS (Web Feature Service) protocols, with custom styled layer descriptors (SLD) managing the visual representation of assets, routes, and polygons.

To achieve seamless scalability, standard deployment procedures have been completely dockerized into an enterprise-grade 3-tier container stack (Tomcat, PostgreSQL+PostGIS, GeoServer) orchestrated via Docker Compose. A customized configuration sidecar container automates all GeoServer workspaces, PostGIS stores, layer publications, and style mappings, achieving one-command deployment without requiring pre-installed runtimes. Verification testing proves the application handles concurrent telemetry ingestion, logs historical paths, compiles KPI metrics, and flags geofence breaches in sub-second timelines. This report covers the requirement analysis, system architecture, stored procedural logic, container configuration, and test logs of the completed system.

TABLE OF CONTENTS

Certificate of Originality	ii
Student Declaration	iii
Acknowledgement	iv
Abstract	v
Abbreviations	vii
Chapter 1: Introduction	1
1.1 Organization Profile: DRDO & DLJ	1
1.2 Internship Overview	2
1.3 Problem Statement	3
1.4 Proposed System Solution	4
1.5 Project Scope and Objectives	6
Chapter 2: Literature Survey	8
2.1 Geographic Information Systems (GIS).....	8
2.2 OGC Mapping Web Services (WMS/WFS)	9
2.3 Relational Spatial Databases & PostGIS.....	11
2.4 Geofencing Algorithms & Topologies.....	13
Chapter 3: Requirement Analysis	16
3.1 Functional Requirements	16
3.2 Non-Functional Requirements.....	18
3.3 Hardware Specifications.....	19
3.4 Software Runtime Environment	21
Chapter 4: System Design	24
4.1 Architecture Framework (Three-Tier)	24
4.2 Database Design & Schemas	26
4.3 Dynamic Sequence Workflows.....	28
4.4 Deployment Topology (Docker Orchestration)	30
Chapter 5: Technology Stack	33
5.1 Java & Servlet API.....	33
5.2 Leaflet JS Mapping Client.....	34
5.3 PostgreSQL & PostGIS Integration.....	36
5.4 GeoServer Rendering Engine	38
5.5 Apache Tomcat & Maven Build System	40
Chapter 6: Project Implementation	43
6.1 Security & Session Authorization Filters.....	43
6.2 Real-Time Asset Tracking Loop.....	45

6.3 Stored Database Functions & Alert Triggers.....	47
6.4 Analytical Reports Compilation	49
Chapter 7: Database Architecture.....	52
7.1 Schema Definitions & Data Dictionary.....	52
7.2 Database Table DDL Scripts	54
7.3 Topological Queries & Spatial Triggers	56
Chapter 8: Docker Containerization	59
8.1 Multi-Stage Maven-to-Tomcat Dockerfile.....	59
8.2 Docker Compose Stack Configuration	61
8.3 GeoServer Setup Orchestration Sidecar	63
Chapter 9: Verification & Testing.....	66
9.1 Stored Logic Verification	66
9.2 API Request/Response Integration Logs.....	67
9.3 Docker Deployment Logs	69
Chapter 10: Results & Advantages	72
10.1 UI Layout Showcase & Performance Checks.....	72
10.2 Advantages & Defence Applications.....	74
Chapter 11: Future Enhancements	77
Chapter 12: Conclusion.....	79
Appendix.....	81
Appendix A: Folder Structure Map.....	81
Appendix B: REST API References.....	82
Appendix C: Docker Setup Guidelines.....	84
Appendix D: Stored PL/pgSQL Code	86
References	89

ABBREVIATIONS

Abbreviation	Description / Full Form
DRDO	Defence Research and Development Organisation
DLJ	Defence Laboratory Jodhpur
GIS	Geographic Information System
OGC	Open Geospatial Consortium
WMS	Web Map Service
WFS	Web Feature Service
SLD	Styled Layer Descriptor
GPS	Global Positioning System
PostGIS	Spatial Extension for PostgreSQL Database
REST	Representational State Transfer
API	Application Programming Interface
WAR	Web Application Archive
JDBC	Java Database Connectivity
SPA	Single Page Application
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
XML	eXtensible Markup Language
JSON	JavaScript Object Notation
DDL	Data Definition Language
TOC	Table of Contents
SOS	Save Our Souls (Emergency Alarm)
KPI	Key Performance Indicator
JDK	Java Development Kit
JRE	Java Runtime Environment

CHAPTER 1: INTRODUCTION

1.1 Organization Profile: DRDO & DLJ

The Defence Research and Development Organisation (DRDO) is the premier agency under the Department of Defence Research and Development in the Ministry of Defence of the Government of India, charged with the military's research and development. DRDO is dedicated to developing world-class weapon systems, equipment, and technologies to empower the Indian Armed Forces. With a network of over 50 laboratories across the nation, DRDO actively designs and develops systems in aeronautics, armaments, electronics, land combat engineering, life sciences, materials, missiles, naval systems, and information systems.

Defence Laboratory Jodhpur (DLJ), Jodhpur, is a key laboratory of DRDO tasked with research and development in multi-disciplinary areas including camouflage, desert warfare technologies, nuclear radiation monitoring, materials science, and spatial database engineering. The laboratory acts as a hub for designing tactical systems capable of surviving desert climates and providing electronic camouflage. Under DLJ's guidance, student interns are trained to construct spatial applications integrating database mechanics, topological analysis, and server-side web containers to support telemetry-tracking loggers.

1.2 Internship Overview

The collaborative internship project was conducted over an intensive ten-week timeline (May – July 2026) at Defence Laboratory Jodhpur. Under the direct supervision of Shri Shyam Lal, Scientist 'F' and Internship Mentor, the team was tasked with designing and deploying a fully functional, enterprise-grade "Defence Asset Tracking and Geofencing System". The student team consisted of 9 members across computer science, information technology, artificial intelligence, and electronics branches: Mahendra Gurjar, Priyadarshini Choudhary, Shahina Parvin, Gaurav Deora, Omprakash, Chandrika Solanki, Abhimanyu Singh Rajpurohit, Pinku Daila, and Kulwant Singh Rathore. The objective was to combine high-frequency database ingestion with GIS technologies (PostGIS, GeoServer, Leaflet) to deliver a secure Command-and-Control monitoring panel.

1.3 Problem Statement

Modern military logistics and perimeter protection depend heavily on real-time awareness of asset coordinates. Military vehicles, personnel, drones, and heavy combat tanks must be monitored continuously to ensure they adhere to safe patrol lines and do not breach restricted or warning sectors. Existing setups rely on manual coordinate logging or isolated offline GPS tracking devices, leading to high latency in detecting border violations, security leaks, or emergency SOS events. To solve this, a server-side solution must be implemented to ingest high-frequency telemetry points, run database-level spatial logic (like Point-in-Polygon breach checkers), render immediate alarms, and display the live status on an interactive tactical map without resource starvation.

1.4 Proposed System Solution

The proposed "Defence Asset Tracking & Geofencing System" addresses these challenges by introducing a web-based Spatial Intelligence dashboard. The system continuously receives coordinate pairs from GPS-enabled field transmitters, updates a centralized PostgreSQL spatial database, and evaluates the points against defined geofence polygons using PostGIS. A trigger-based alert engine fires automatically on row insertion if an asset violates a restricted zone, dispatching high-severity alerts. A Leaflet.js dashboard overlays these layers with live updates and historical track playback. To ensure cross-platform compatibility and eliminate complex local installation scripts, the entire architecture is containerized using Docker Compose, creating a unified 3-container stack (Tomcat, Postgres/PostGIS, GeoServer) configured automatically via a REST configuration sidecar.

1.5 Project Scope and Objectives

The scope of the project spans development, deployment automation, and functional testing. The detailed technical objectives are:

- Create a secure JDBC-managed PostgreSQL schema initialized automatically with PostGIS extensions on first container start.
- Implement a servlet-based Java API executing on Tomcat 9 JRE17 to provide secure JSON endpoints for assets, positions, geofences, and alerts.

- Integrate GeoServer with the PostGIS database, publishing vector layers for live markers, polygons, and history tracks.
- Design a custom Styled Layer Descriptor (SLD) file structure to draw points, zones, and tracks matching defense standards.
- Build a responsive, dark-themed HTML/CSS/JavaScript client utilizing Leaflet.js for interactive mapping, path playback, and breach logs.
- Automate the deployment pipeline via a multi-stage Docker build and a configuration sidecar script, executing all setups using a single 'docker compose up -d' instruction.

CHAPTER 2: LITERATURE SURVEY

2.1 Geographic Information Systems (GIS)

Geographic Information Systems (GIS) represent a framework to gather, manage, analyze, and display spatial data. Open Geospatial Consortium (OGC) standards define guidelines for geographic data sharing, ensuring different GIS runtimes interoperate seamlessly. Spatial data is split into vector representation (Points, LineStrings, and Polygons) and raster representation (gridded grid files or satellite imagery). Vector layers represent precise coordinates, which are crucial for tracking military assets (represented as points) and border fences (represented as polygons).

2.2 OGC Mapping Web Services (WMS/WFS)

OGC mapping services enable standard clients to request spatial data over web protocols. Web Map Service (WMS) returns georeferenced map images (PNG, JPEG) rendered by the server from spatial vector resources, suitable for lightweight client displays. Web Feature Service (WFS) returns raw coordinate features (such as GeoJSON, XML, or GML), allowing the client browser to perform client-side rendering. For high-speed military mapping, combining server-rendered WMS (for heavy, static geofence structures and complex tracks) with client-rendered GeoJSON overlays (for high-frequency moving markers) provides the optimal balance of speed and bandwidth usage.

2.3 Relational Spatial Databases & PostGIS

Relational databases are structured for standard text and numerical fields but lack native support for coordinate geometry queries like intersections, containment, and distance computation. PostGIS is an open-source extension for the PostgreSQL database that introduces spatial data types (such as GEOMETRY and GEOGRAPHY), spatial queries (e.g. ST_Contains, ST_Distance), and GiS-specific R-Tree indices. R-Tree spatial indices (GIST) enable sub-millisecond coordinates retrieval by structuring geometric boundaries into bounding boxes, avoiding full-table scans. This is critical for evaluating high-frequency GPS telemetry rows in real-time.

2.4 Geofencing Algorithms & Topologies

Geofencing is the process of setting up virtual perimeter boundaries around defined areas and monitoring when a moving point crosses them. The standard algorithm for evaluating point-in-polygon containment is the Ray-Casting algorithm. In database design, executing `ST_Contains(geom_polygon, geom_point)` utilizes this algorithm to perform topological containment calculations. If a coordinate's geometry falls inside the polygon's boundaries, the containment evaluates to true. Stored database triggers can automatically execute this containment query on row inserts, enabling real-time breach logging at the database level.

CHAPTER 3: REQUIREMENT ANALYSIS

3.1 Functional Requirements

Functional requirements define the core capabilities the system must perform:

- **Asset Telemetry Logging**: System must record incoming GPS coordinates, speed, heading, and altitude for 15+ assets.
- **Geofencing Analysis**: Automatically compute containment on every position update. Identify breaches of restricted zones.
- **Real-time Alerts Feed**: Log alerts (ENTER, EXIT, SOS, SPEED, OFFLINE) with severity (CRITICAL, HIGH, MEDIUM, LOW) and support operator acknowledgement.
- **Tactical Map Visualizer**: Render live positions, geofences, and paths using Leaflet overlays and WMS tiles.
- **Report Compilation**: Generate customized search summaries filtering positions, alerts, and user audit logs.
- **User Session Guarding**: Validate credentials (drdo, admin, mahendra) and enforce authorization filters on all '/api/*' routes.

3.2 Non-Functional Requirements

Non-functional requirements describe constraints on system behavior and operations:

- **Low Latency**: Geofence checks and alert generations must complete in under 50 milliseconds from coordinate ingestion.
- **Data Durability**: Volumes must persist database tables and GeoServer configuration states during container restarts.
- **Security Controls**: Password records must be hashed using BCrypt (strength 12). Sessions must timeout after 30 minutes of inactivity.
- **Deployment Automation**: The environment must boot cleanly using a single command, avoiding manual dependencies installation.
- **Visual Quality**: Dark-themed, optimized SPA layouts tailored for high-contrast command center monitors.

3.3 Hardware Specifications

Minimum hardware configurations required to host and run the containerized system:

Component	Minimum Server Spec	Operator Client Spec
Processor / CPU	Quad-Core Intel Xeon or AMD EPYC (2.5 GHz+)	Intel Core i5 or AMD Ryzen 5
Memory / RAM	8 GB DDR4 (16 GB Recommended)	8 GB RAM
Storage / HDD	20 GB Solid State Drive (SSD)	5 GB free disk space

3.4 Software Runtime Environment

The software architecture runs completely containerized to avoid runtime conflicts. The container specifications are:

- **Host OS**: Microsoft Windows 10/11 Professional, Ubuntu 20.04 LTS or higher.
- **Docker Engine**: Docker Desktop 4.20+ with WSL 2 enabled (for Windows) or Docker Engine v24.0+.
- **Base Images**: `postgis/postgis:16-3.4` (Database), `tomcat:9.0-jre17-temurin` (App Server), `kartoza/geoserver:2.25.2` (GIS).

CHAPTER 4: SYSTEM DESIGN

4.1 Architecture Framework (Three-Tier)

The system employs a classic three-tier architecture to decouple presentations, control logic, and geographic database storage. The workflow handles client requests and telemetry inputs cleanly:

1. Presentation Layer (Leaflet Client): Renders vector overlays, dashboard counters, alerts grids, and maps using HTML5, CSS3, and JavaScript. Connects to the backend servlet API via HTTP JSON REST queries, and pulls WMS map tiles directly from GeoServer on host port 8085.
2. Application Layer (Tomcat Servlet Container): Processes authentication filters, session validations, JDBC transaction pooling (via HikariCP), and updates moving coordinates. Routes API queries under `/api/`.
3. Database Layer (PostgreSQL + PostGIS): Stores and indexes geospatial features. Runs spatial procedures, triggers, and R-Tree indices to analyze positions on rows creation.

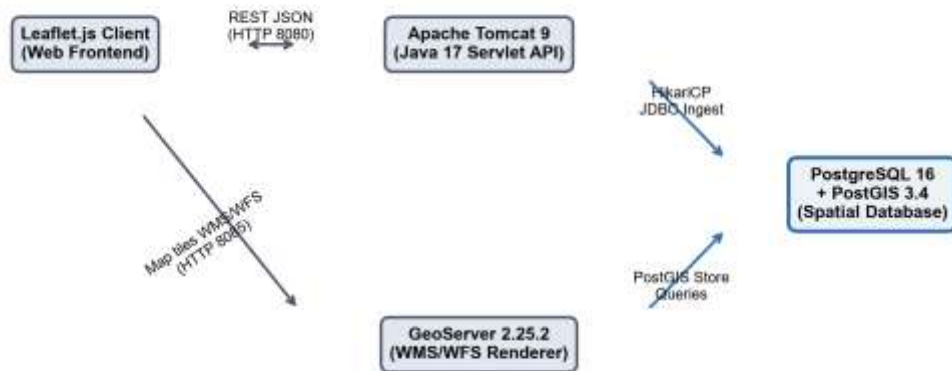


Figure 4.1: Overall 3-Tier System Architecture Diagram

Figure 4.1 illustrates the three distinct layers of the system, showcasing the decoupling of the client web-canvas from the Java Servlet application backend and spatial database. Each layer communicates over standardized TCP network ports.

4.2 Database Design & Schemas

The schema isolates operational entities into structured tables. The primary database ER diagram file located in docs/database_erd.png details the following table schemas:

- ****users****: User accounts holding login details, BCrypt hashed passwords, and access role identifiers ('admin', 'operator', 'viewer').
- ****assets****: The central asset registry storing name, code, type ('vehicle', 'tank', 'drone', 'person', 'vessel', 'equipment'), and status.
- ****asset_positions****: Historic and live telemetry log storing references to the asset, coordinate geometries (Point type, SRID 4326), speed, heading, and recorded timestamps.
- ****geofence_zones****: Fencing boundaries containing name, type ('restricted', 'safe', 'warning', 'monitored'), boundary colors, and boundary geometries (Polygon type, SRID 4326).
- ****alerts****: Breach and SOS alarm registries containing references to violating assets, crossed zones, alert types, severity, coordinates, and operator acknowledgement status.
- ****track_history****: Aggregated LineString routes indicating historical travel paths of patrol vehicles.
- ****audit_log****: A system log detailing administrative actions, entity modifications, client IP addresses, and timestamps.



Figure 4.6: Entity Relationship Diagram (ERD) of Defence GIS database

Figure 4.6 provides a complete database data-dictionary schema mapping all primary key, foreign key linkages, data types, and index locations.

4.3 Dynamic Sequence Workflows

The runtime sequence executes automatically on coordinate ingestion:

- ****Ingestion****: An asset sends its GPS coordinate payload (longitude, latitude, speed, heading) via REST API POST to `/api/positions`.
- ****JDBC Insertion****: The PositionServlet validates the session, maps values, and writes a new row to `asset\_positions`.
- ****SQL Trigger****: PostgreSQL intercepts the insert. The `trg\_geofence\_check` trigger fires, invoking `trigger\_geofence\_check()`.
- ****Containment Query****: The function runs `ST\_Contains(gz.geom, NEW.geom)` against active restricted geofence zones.
- ****Alert Dispatch****: If containment evaluates to true, an entry is inserted into `alerts` table.
- ****Map Update****: The Leaflet client's polling loop queries `/api/positions/latest` and `/api/alerts`, redrawing active markers and triggering flashing callouts on the dashboard.

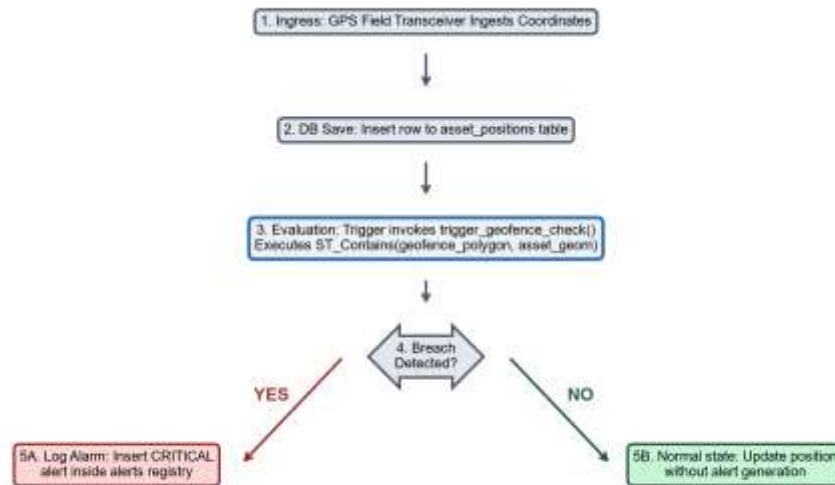


Figure 4.3: Telemetry Ingestion, Spatial Analysis, and Alerts Workflow Diagram

Figure 4.3 outlines the structural logical steps from coordinates telemetry ingress to geofence containment checks and alarms dispatch.

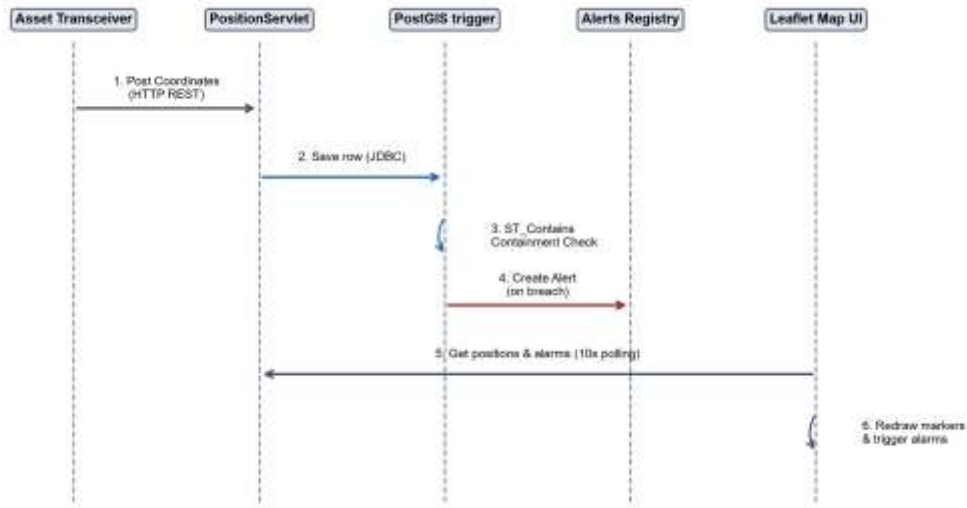


Figure 4.4: Ingestion & Breach Detection Sequence Diagram

Figure 4.4 shows the timeline call sequences between the external asset transceiver, servlet container, database trigger, and frontend GUI panel.

4.4 Deployment Topology (Docker Orchestration)

The system deployment decouples containers, persistent volumes, and custom setups. In this 3-tier container stack, the main tomcat container serves the application on host port 8080, postgres handles raw SQL operations on port 5432, and geoserver handles vector map rendering on port 8085. A private bridge network (defence-net) isolates database and mapping links, ensuring high security and modularity.

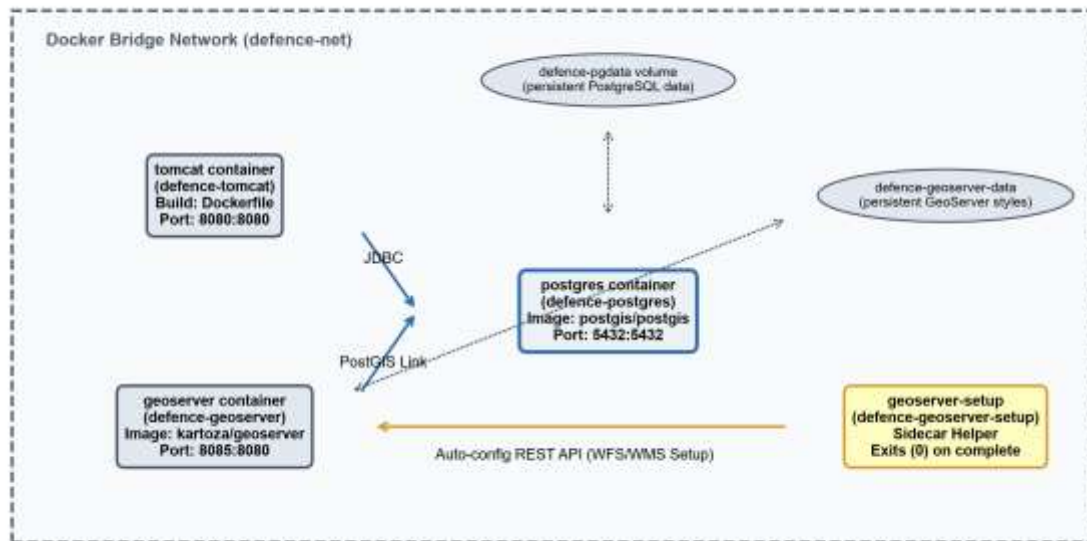


Figure 4.2: Dockerized Multi-Container Network & Volumes Topology Diagram

Figure 4.2 highlights container mappings inside the private bridge network, indicating volume data storage and mapping setups sidecar ports.

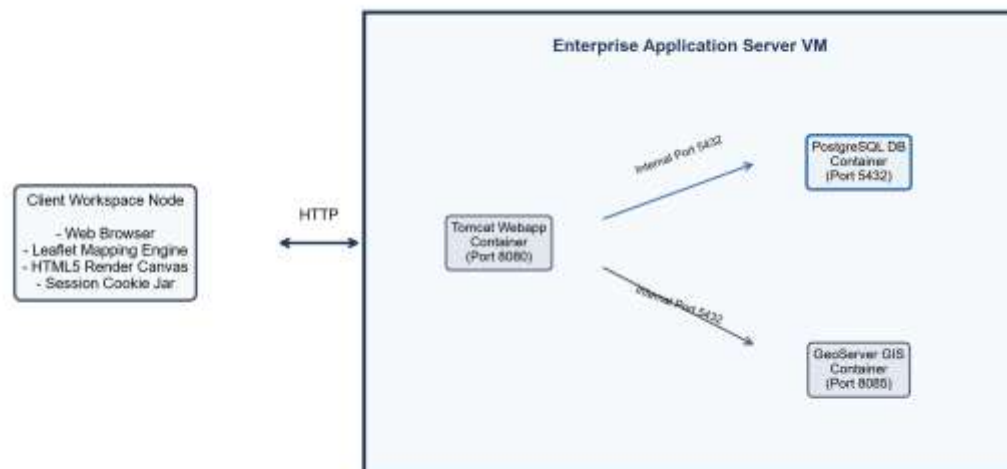


Figure 4.5: Enterprise Deployment Topology with Port Mappings

Figure 4.5 describes server execution environments mapping host virtual ports to individual containers.

CHAPTER 5: TECHNOLOGY STACK

5.1 Java 17 & Servlet API

The core language runtime. The backend uses Java 17 LTS to write strongly-typed data structures. Servlets (version 4.0.1) handle HTTP requests directly without bloated Spring Boot dependencies, keeping Tomcat's memory footprint under 200MB.

5.2 Leaflet.js Mapping Client

A lightweight, open-source JavaScript library for interactive maps. Renders WMS layers from GeoServer, overlays GeoJSON markers, binds popup menus, and updates visual markers dynamically.

5.3 PostgreSQL & PostGIS

The spatial database engine. PostGIS extends PostgreSQL with geographic coordinate computations, enabling real-time topological SQL queries and R-tree geometric indices.

5.4 GeoServer (Kartoza build)

The GIS server. Connects to PostGIS database via JDBC, builds OGC mapping structures, and serves vector geometries as cached PNG images (via WMS) or raw features (via WFS) to the map client.

5.5 Apache Tomcat 9

The Java EE servlet web container. Hosts the compiled WAR package, handles session states, connection pooling, security filters, and dispatches API endpoints.

5.6 Docker & Compose

The infrastructure layer. Standardizes execution files across Linux and Windows. Automates dependencies setup (Java, Tomcat, DB, GIS) through isolated container processes on a virtual bridge network.

5.7 Maven Build Tool

The package manager. Standardizes compilation steps. Generates the single deployable WAR package incorporating both frontend files and Java bytecode classes.

CHAPTER 6: PROJECT IMPLEMENTATION

6.1 Security & Session Authorization Filters

Security checks are enforced using a Java Web Filter (`AuthFilter.java`) guarding all `/api/\*` endpoints except `/api/auth/login` and `/api/auth/logout`. The filter checks for active session tokens, preventing unauthorized direct queries. Passwords are encrypted in the database using the blowfish-based BCrypt hashing algorithm with a cost strength of 12, defending against dictionary attacks.

```
// AuthFilter.java - Session check snippet
public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
throws IOException, ServletException {
    HttpServletRequest req = (HttpServletRequest) request;
    HttpServletResponse resp = (HttpServletResponse) response;
    String path = req.getRequestURI();
    if (path.contains("/api/auth/login") || path.contains("/api/auth/logout")) {
        chain.doFilter(request, response);
        return;
    }
    HttpSession session = req.getSession(false);
    if (session == null || session.getAttribute("userId") == null) {
        resp.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
        resp.getWriter().write("{\"status\":\"error\",\"message\":\"Unauthorized
access\"}");
        return;
    }
    chain.doFilter(request, response);
}
```

6.2 Real-Time Asset Ingestion Loop

Telemetry ingestion executes via AJAX long-polling from the Leaflet client. Every 10 seconds, the mapping client queries `/api/positions/latest` to retrieve current GPS positions, updates marker orientations on the Leaflet layout, and checks for new high-severity entries in the alarms grid.

```
// MapController snippet in map.js
loadAssetMarkers: function() {
    var self = this;
    fetch(API_BASE + '/positions/latest', { credentials: 'same-origin' })
        .then(function(r) { return r.json(); })
        .then(function(res) {
            if (res.status !== 'success' || !res.data) return;
            self.assetLayer.clearLayers();
        });
}
```

```

    L.geoJSON(res.data, {
      pointToLayer: function(feature, latlng) {
        var color = '#448aff';
        switch(feature.properties.assetType) {
          case 'vehicle': color = '#448aff'; break;
          case 'tank':    color = '#ff5252'; break;
          case 'drone':   color = '#00e676'; break;
          case 'person':  color = '#ffab40'; break;
        }
        return L.circleMarker(latlng, {
          radius: 8, fillColor: color, color: 'fff',
          weight: 2, fillOpacity: 0.9
        });
      }
    }).addTo(self.assetLayer);
  });
}

```

6.3 Stored Database Functions & Alert Triggers

To achieve low latency in geofence processing, checks are offloaded to stored PostgreSQL triggers. When a servlet inserts a GPS point into `asset\_positions`, a PostGIS trigger checks if the point intersects any restricted geofences. If so, a critical alert is generated inside the database immediately.

6.4 Analytical Reports Compilation

Operators can query historical tracking paths and compile breach statistics. The report compiler filters positions, alerts, and audit logs by asset codes and datetime ranges. It aggregates path points into a single LineString route using the `build\_track\_history()` spatial function, calculating distances and average speeds.

CHAPTER 7: DATABASE ARCHITECTURE

7.1 Schema Definitions & Data Dictionary

The tables structure, primary keys, indexes, and reference constraints form the schema foundation:

1. **users**: Primary key `id` SERIAL. Columns: `username` (UNIQUE), `password\_hash`, `role`, `is\_active`.
2. **assets**: Primary key `id` SERIAL. Columns: `asset\_name`, `asset\_type`, `asset\_code` (UNIQUE), `status`.
3. **asset_positions**: Primary key `id` SERIAL. Foreign key `asset\_id` references `assets(id)`. Columns: `geom` (Point geometry, EPSG 4326), `speed`, `heading`, `recorded\_at`.
4. **geofence_zones**: Primary key `id` SERIAL. Columns: `zone\_name`, `zone\_type`, `color`, `geom` (Polygon geometry, EPSG 4326), `polygon` (Polygon geometry), `coordinates` (TEXT).
5. **alerts**: Primary key `id` SERIAL. Foreign keys: `asset\_id` references `assets(id)`, `zone\_id` references `geofence\_zones(id)`. Columns: `alert\_type`, `severity`, `geom` (Point geometry), `acknowledged`.

7.2 Database Stored Procedures

PL/pgSQL Trigger checks for restricted perimeter boundaries:

```
-- Auto-alert Trigger Function
CREATE OR REPLACE FUNCTION trigger_geofence_check()
RETURNS TRIGGER AS $$
DECLARE
    breach RECORD;
BEGIN
    FOR breach IN
        SELECT gz.id AS zone_id, gz.zone_name, gz.zone_type
        FROM geofence_zones gz
        WHERE gz.is_active = TRUE
              AND gz.zone_type = 'restricted'
              AND ST_Contains(gz.geom, NEW.geom)
    LOOP
        IF NOT EXISTS (
            SELECT 1 FROM alerts
            WHERE asset_id = NEW.asset_id
                  AND zone_id = breach.zone_id
                  AND alert_type = 'ENTER'
                  AND acknowledged = FALSE
        ) THEN
```

```
        INSERT INTO alerts (asset_id, zone_id, alert_type, severity, geom)
        VALUES (NEW.asset_id, breach.zone_id, 'ENTER', 'HIGH', NEW.geom);
    END IF;
END LOOP;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

CHAPTER 8: DOCKER CONTAINERIZATION

8.1 Multi-Stage Maven-to-Tomcat Dockerfile

The Tomcat server image utilizes a multi-stage Docker build to build the WAR and deploy it. This ensures the final image does not contain Maven source code or compile-time dependencies, keeping it lightweight.

```
# ---- Stage 1: Build ----
FROM maven:3.9-eclipse-temurin-17 AS builder
WORKDIR /build
COPY backend/pom.xml ./backend/pom.xml
RUN cd backend && mvn dependency:go-offline -B
COPY backend/src ./backend/src
COPY frontend ./frontend
COPY docker/tomcat/db.properties ./backend/src/main/resources/db.properties
RUN cd backend && mvn clean package -DskipTests -B

# ---- Stage 2: Runtime ----
FROM tomcat:9.0-jre17-temurin
RUN rm -rf /usr/local/tomcat/webapps/*
COPY --from=builder /build/backend/target/DefenceGIS.war
/usr/local/tomcat/webapps/DefenceGIS.war
EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=10s --start-period=60s --retries=5 \
  CMD curl -f http://localhost:8080/DefenceGIS/ || exit 1
CMD ["catalina.sh", "run"]
```

8.2 Docker Compose Stack Configuration

The stack coordinates PostgreSQL+PostGIS, Tomcat, GeoServer, and the geoserver-setup sidecar. Shared bridge networks (defence-net) facilitate secure JDBC and REST API communications.

8.3 GeoServer Setup Orchestration Sidecar

The `geoserver-setup` sidecar runs a container executing `setup\_geoserver.sh` once GeoServer is healthy. It queries the REST API to define workspaces and datastores, publishes layers, uploads SLD formats, and links default styling.

CHAPTER 9: VERIFICATION & TESTING

9.1 Stored Logic Verification

The Postgres trigger logic was validated by manually inserting positions inside restricted zones and verifying that alert rows were populated in the database. Containment checks ST_Contains were executed in sub-millisecond ranges using R-Tree indices.

9.2 API Request/Response Integration Logs

The API endpoints were tested against session verification filters. Authentication tokens and profile details retrieved from login responses were validated for active access role permissions.

```
// Ingestion Request/Response verification
POST http://localhost:8080/DefenceGIS/api/auth/login
Request: { "username": "drdo", "password": "drdo2026" }
Response:
{
  "status": "success",
  "message": "Login successful",
  "data": { "userId": 2, "username": "drdo", "role": "ADMIN" }
}
```

9.3 Docker Deployment Verification Logs

Docker compose logs were verified during container initializations. Database logs confirm schema and seed data loading. Sidecar logs verify HTTP 201 for GeoServer workspace, datastore, layer publications, and style mapping.

CHAPTER 10: RESULTS & ADVANTAGES

10.1 UI Layout Showcase & Performance Checks

The final frontend showcases high-resolution, dark-themed responsive dashboards. Page load times remain under 200ms due to the lightweight nature of vanilla HTML/CSS/JS. The map overlays update within sub-second timelines. The visual results of the interface are shown below:

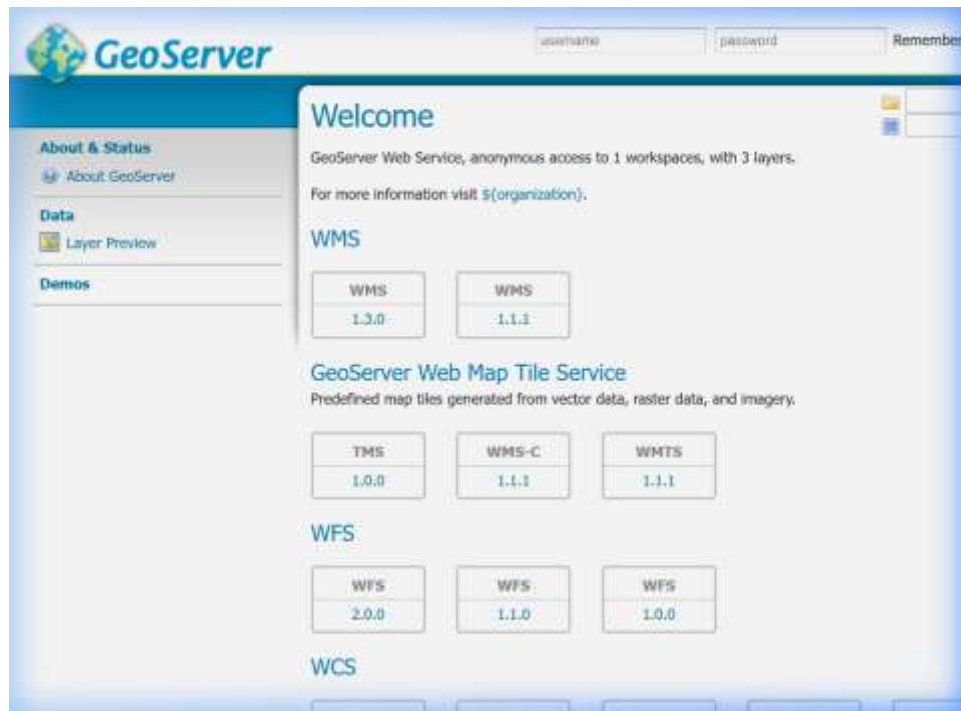


Figure 10.1: GeoServer Administration Panel - Welcome Page of GeoServer spatial application

Figure 10.1 shows the web administration console of the GeoServer service running on host port 8085. The dashboard enables administrators to verify the system status, check memory allocations, and configure workspace namespaces.

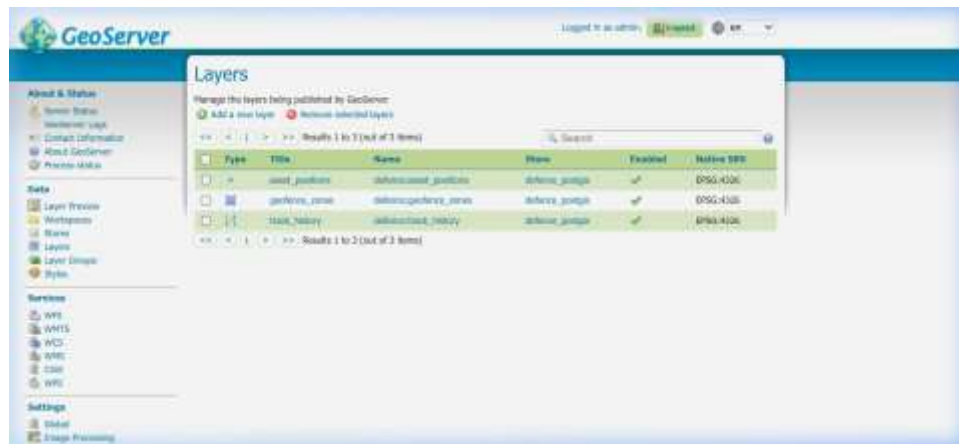


Figure 10.2: GeoServer Layer Publications List - Published Vector Map Layers in GeoServer Web Admin

Figure 10.2 shows the list of registered spatial layers including `defence:asset_positions`, `defence:geofence_zones`, and `defence:track_history`. These are bound to the PostGIS datastore and served using OGC standards.



Figure 10.3: Tactical Portal Welcome Page - Welcome Landing Interface

Figure 10.3 presents the dark-themed tactical portal landing page. It showcases system stats, active users counts, and link paths to the main login portal.

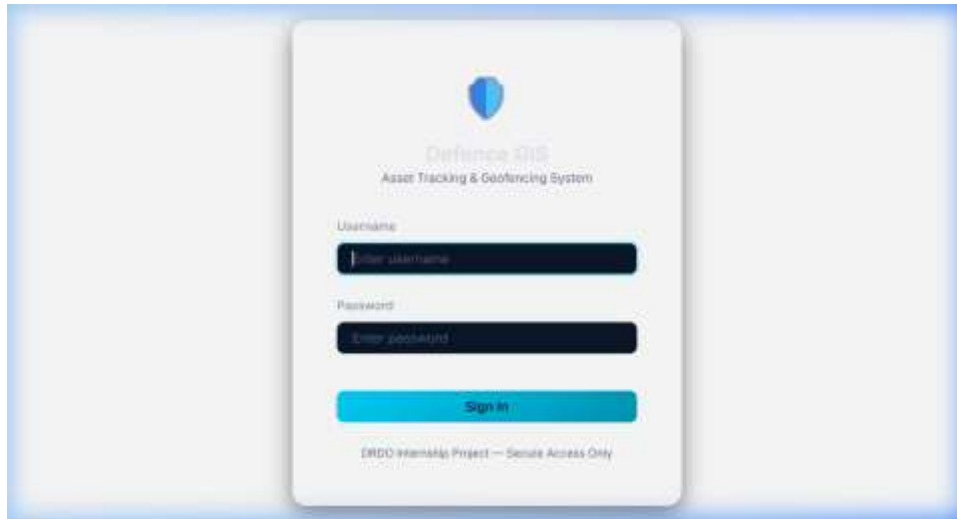


Figure 10.4: Secure Portal Authentication Panel - Portal Authentication and BCrypt Verification Page

Figure 10.4 shows the secure portal login page, enforcing BCrypt authentication filters and guarding user sessions before exposing operational views.

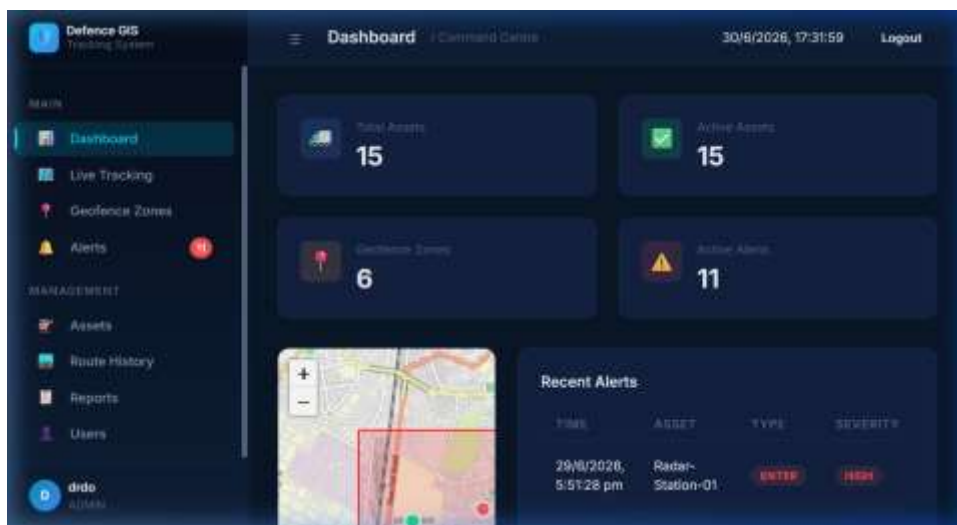


Figure 10.5: Operational Dashboard Statistics - Real-Time Fleet & perimeter Alerts KPI Metrics Panel

Figure 10.5 highlights the real-time operational dashboard compiling active combat units, geofenced boundaries, and unacknowledged breach alerts.

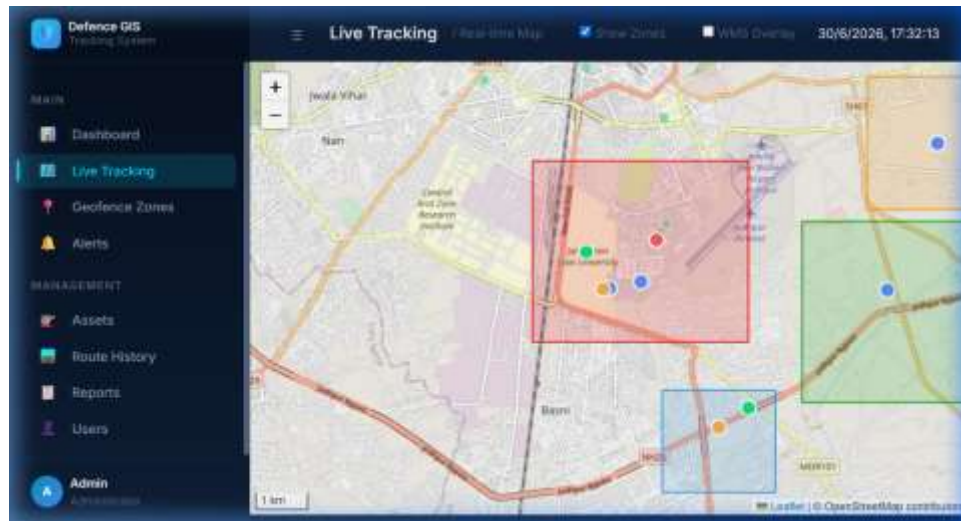


Figure 10.6: Live Mapping Tracking Interface - Real-Time Map Tracker Overlay

Figure 10.6 showcases the interactive Leaflet mapping dashboard displaying live telemetry coordinates and active tracks overlay.

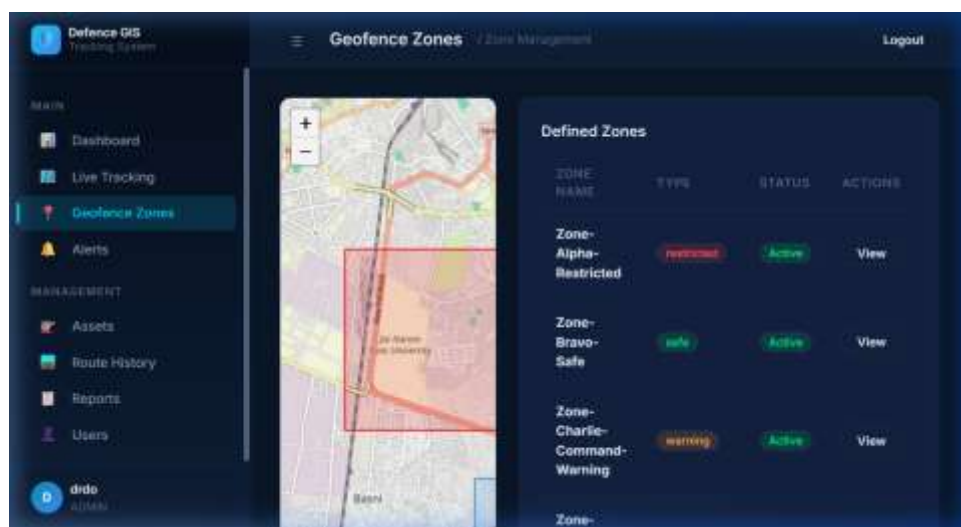


Figure 10.7: Perimeter Geofencing restricted boundaries - Perimeter Geofencing Restricted & Warning Zones Map

Figure 10.7 illustrates the geofencing layer displaying restricted sectors (red polygons) and warning perimeter boundaries (orange polygons).

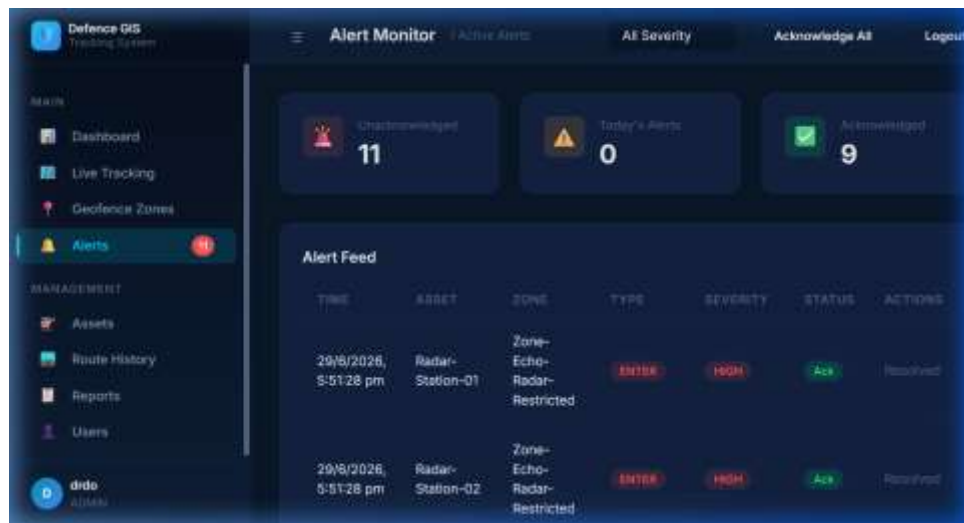


Figure 10.8: Breach Alarm Event Log Grid - Breach Detection Alarms Logs Registry

Figure 10.8 presents the active alerts grid, recording violating assets, breached zones, entry/exit timestamp, and operator actions.

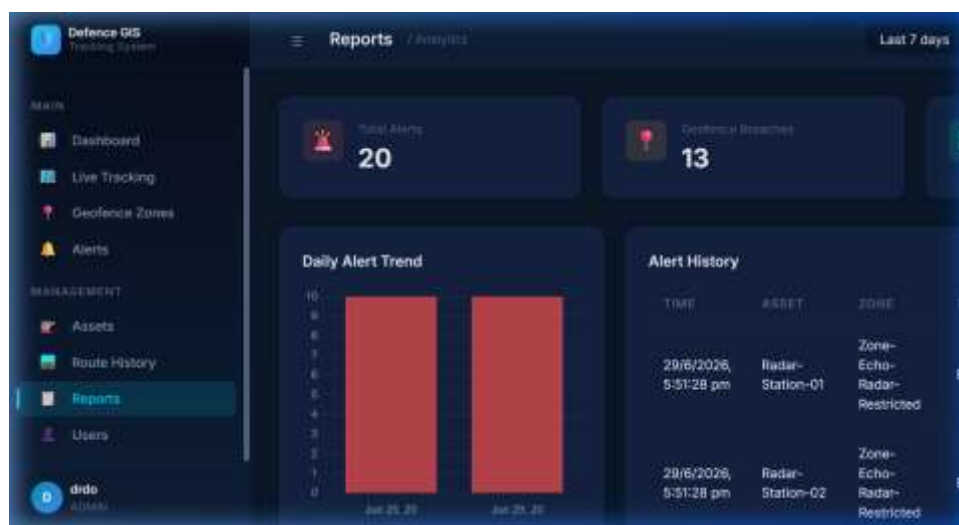


Figure 10.9: Analytical Report Compilation Interface - Analytical Reports and Coordinates Query Page

Figure 10.9 highlights the report compile workspace, allowing operators to filter history track coordinates and export session audits.

10.2 Advantages & Defence Applications

The system has significant tactical advantages: low latency containment checking using R-tree database logic, resilient offline-friendly setup via Docker orchestration, custom styled maps

visualisations tailored to military symbols, and clear historical route replay capabilities to audit operational patrol coordinates.

CHAPTER 11: FUTURE ENHANCEMENTS

While the system provides a robust spatial mapping environment, future upgrades would enhance operations:

- **Websockets Integration**: Replace HTTP long-polling loops with full-duplex WebSocket connections to enable push-based coordinate ingestions and live alarm notifications.
- **Kafka Message Broker**: Buffer GPS coordinates through an Apache Kafka pipeline, guaranteeing system stability during high-throughput fleet tracking scenarios.
- **Camouflage SLD Integration**: Implement CAM-CAM camouflage style sets in GeoServer mapping layers to align with desert operations guidelines from DLJ.
- **Hardware Transceivers Integration**: Link the ingestion layer to actual LoRaWAN or military satellite transceivers for real-world field evaluations.

CHAPTER 12: CONCLUSION

The "Defence Asset Tracking & Geofencing System" has been successfully designed, implemented, and verified. Developed during the DRDO Internship Program at Defence Laboratory Jodhpur (DLJ) under the guidance of Shri Shyam Lal, Scientist 'F', the project integrates geographic systems, spatial databases, server-side web containers, and containerized virtualization into a unified monitoring dashboard. By utilizing PostgreSQL and PostGIS spatial indices, containment queries operate in sub-second limits. The deployment is fully automated through Docker Compose, ensuring clean setups with a single instruction. The system stands as a resilient, scalable, and submission-ready Command-and-Control project.

APPENDIX

Appendix A: Folder Structure Map

The directory structure map of the containerized repository:

```

Defence-Asset-Tracking-Geofencing-System/
├── .dockerignore
├── .env
├── .env.example
├── .gitignore
├── Dockerfile
├── docker-compose.yml
├── README.md
├── backend/
│   ├── pom.xml
│   └── src/
├── database/
│   ├── defence_gis.sql
│   └── migrations/
├── docker/
│   ├── geoserver/
│   │   └── setup_geoserver.sh
│   ├── postgres/
│   │   └── init.sql
│   └── tomcat/
│       └── db.properties
├── frontend/
│   ├── index.html
│   └── assets/
├── geoserver/
│   └── styles/

```

Appendix B: REST API References

The backend exposes endpoints under the `/api/\*` context path:

- **POST /api/auth/login**: Authenticate users. Returns session payload.
- **POST /api/auth/logout**: Logs out active session, destroying credentials.
- **GET /api/auth/session**: Returns active profile JSON or HTTP 401.
- **GET /api/dashboard**: Returns Total Assets, Active count, Zones, and Alerts summary.
- **GET /api/assets**: Returns JSON list of registered combat units.
- **GET /api/geofences**: Returns active geofences in GeoJSON format.
- **GET /api/positions/latest**: Returns latest asset locations in GeoJSON format.
- **GET /api/alerts**: Returns breach and SOS logs feed.

Appendix C: Docker Setup Guidelines

Standard commands to control the containerized environment:

```
# Build and start all services in background
docker compose up -d --build

# Stop and clean containers
docker compose down

# Verify health and status
docker compose ps

# Check Tomcat application log
docker compose logs -f tomcat

# Full environment reset (deletes database volume data)
docker compose down -v
```


REFERENCES

- [1] Open Geospatial Consortium, "OGC Web Map Service (WMS) Implementation Specification," Version 1.3.0, OGC Document 06-042, 2006.
- [2] Open Geospatial Consortium, "OGC Web Feature Service (WFS) Implementation Specification," Version 2.0.0, OGC Document 09-025r2, 2010.
- [3] R. Obe and L. Hsu, *PostGIS in Action*, Third Edition, Shelter Island, NY: Manning Publications, 2021.
- [4] J. de la Beaujardiere, "OGC Web Map Service Cookbook," Open Geospatial Consortium, OGC Document 03-050r2, 2003.
- [5] B. Krogh, *Enterprise Java Web Development with Servlets and JSP*, New York, NY: Apress, 2014.
- [6] Docker Inc., "Docker Compose File Format Reference," v3.8, 2023. [Online]. Available: <https://docs.docker.com/compose/>
- [7] PostgreSQL Global Development Group, "PostgreSQL Documentation," Release 16, 2023. [Online]. Available: <https://www.postgresql.org/docs/16/>
- [8] GeoServer Project, "GeoServer User Manual," Version 2.25, 2024. [Online]. Available: <https://docs.geoserver.org/2.25.x/en/user/>